

The Energy Cost of Execution-Idle in GPU Clusters

Yiran Lei, Jared Fernandez, Vasilis Kypriotis, Dimitrios Skarlatos,
Emma Strubell, Justine Sherry, Daniel Vosler

Abstract

GPUs are becoming a major contributor to data center power, yet unlike CPUs, they can remain at high power even when visible activity is near zero. We call this state *execution-idle*. Using per-second telemetry from a large academic AI cluster, we characterize execution-idle as a recurring low-activity yet high-power state in real deployments. Across diverse workloads and multiple GPU generations, it accounts for 19.7% of in-execution time and 10.7% of energy. This suggests a need to both reduce the cost of execution-idle and reduce exposure to it. We therefore build two prototypes: one uses automatic downscaling during execution-idle, and the other uses load imbalance to reduce exposure, both with performance trade-offs. These findings suggest that future GPU systems should treat execution-idle as a first-class operating state.

1 Introduction

AI’s growing power demand is becoming a significant environmental concern. Data centers consume roughly 4–5% of U.S. electricity and are projected to reach as much as 17% by 2030 [6, 14, 47]. GPUs are a major driver of this trend: they account for about 60% of power in multi-GPU servers [39] and roughly 41% of total power in AI clusters [7]. Yet current understanding of AI energy use is still dominated by coarse aggregate metrics, such as total GPU-hours and Thermal Design Power (TDP) [12, 20, 46]. While these metrics convey the overall scale of energy consumption, they obscure how GPU power evolves during execution. As we show in this paper, a fine grained view of runtime behavior reveals where GPU energy is spent productively and where it is not.

We observe that a GPU can continue drawing substantial power even when a live job shows little compute, memory, or communication activity. This behavior is counterintuitive: one might expect an underutilized GPU to consume relatively little energy, but our measurements show otherwise. In state-of-the-art serving traces, such intervals can account for up to 65% of total energy use. We call this state *execution-idle*: intervals during execution in which the GPU remains allocated and a program remains loaded, yet visible activity is near zero. This differs from truly idle periods, in which the GPU is unused and returns to baseline power.

Execution-idle states are easy to overlook for two reasons. First, CPU-based energy intuition [1, 8] does not carry over cleanly to GPUs. As Figure 1 shows¹, CPU power typically

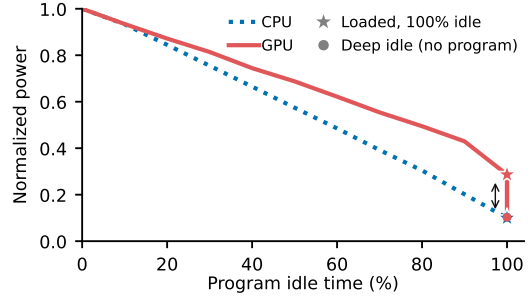


Figure 1. CPU power falls with idle time, but GPU power remains elevated even when a loaded program is fully idle.

tracks inactivity more closely, whereas GPU power can remain elevated even when a loaded program is fully idle. Second, with few exceptions [39, 48], prior work analyzes GPU energy at a coarse granularity, for example through end-to-end summaries or comparisons across different load levels [2–5, 9, 13, 16, 19, 20, 24–26, 39, 40, 43, 46, 49, 50, 58, 59, 62]. What remains missing is a fine-grained characterization of how GPU power and visible activity co-evolve during execution in real deployments.

In this paper, we study execution-idle as a recurring operating regime in modern GPU systems. To do so, we build a passive profiling system that collects per-second GPU power and utilization telemetry. We deploy it for 31 days on a large academic AI cluster. To validate that execution-idle states can occur outside of the university setting, we complement our data from academic, experimental workloads with replays of production, industrial serving workloads. This combined view lets us define execution-idle explicitly, quantify its prevalence and energy cost for a range of settings, and examine causes for why it arises in these settings. All measurements are collected under the cluster’s standard performance-oriented production configuration [15], without power caps or manual tuning.

Our measurements show that execution-idle is a non-trivial component of GPU energy use across workloads, platforms, and serving environments.

- **Execution-idle appears across all six GPU platforms we study.** We observe sustained execution-idle intervals in both training and inference jobs across six GPU types, including NVIDIA B200 [28], showing that this phenomenon is not confined to a particular workload or hardware generation.

¹We run a matrix multiplication benchmark with a configurable pause fraction on an Intel Xeon 6226R and an NVIDIA L40S. CPU power includes package and DRAM power measured with Intel VTune; GPU power is total

device power reported by `nvidia-smi`. The figure excludes other server components such as the motherboard and storage.

- **Execution-idle is especially costly for serving workloads.** Bursty arrivals create loaded-but-inactive gaps, making execution-idle a substantial source of serving energy. It accounts for 48% of energy in long-lived, academic serving workloads and 7–65% across five replays of industry-derived traces from OpenAI [54], Qwen [53], and Azure [49]. This is particularly important because serving is projected to account for up to 70% of energy use in industry clusters [41, 56].
- **Execution-idle also significantly contributes energy cost to non-serving workloads we observed.** Our academic workloads also included training and batched inference workloads, which spent 6% and 7% of total energy consumption in execution-idle states respectively; with 13% and 12% of total execution time in execution-idle states.
- **Many execution-idle intervals are associated with I/O bottlenecks.** By examining the activity immediately preceding execution-idle intervals, we find that they often follow PCIe transfers (48% of cases), network-backed I/O (17%), NVLink communication (2%), and other events, suggesting that execution-idle likely arises from software bottlenecks waiting on I/O.

Together, our measurements show that relatively brief stalls can become a meaningful source of energy cost. From the perspective of hardware researchers, the cost of execution-idle states points to a need for continued research towards power-proportional GPU design. From the perspective of software systems researchers, we must make due with the hardware we have, with mechanisms to either lower the cost or execution-idle periods or reduce exposure to execution-idle periods in the first place.

Unfortunately, naïve approaches to cap GPU energy during execution-idle states (reducing cost) or to increase utilization (reducing the time spent in execution idle states) perhaps predictably save energy, but also increase response latency. To explore whether energy waste due to execution-idle states has an easy fix, we implemented two simple prototypes. One applies automated fine-grained downscaling to reduce power during execution-idle. The other applies deliberate load imbalance in serving to consolidate work onto fewer GPUs, allowing the remaining GPUs to stay idle for longer intervals and avoid execution-idle altogether. Both reduce energy, but both also substantially increase latency, showing that execution-idle is actionable but not free to manage with naïve techniques.

Overall, we argue that execution-idle should be a first-class concern for future energy-efficient GPU systems. Hardware should better approach power proportionality during execution-idle, while software should either keep GPUs more fully utilized or coordinate explicitly with power-management

mechanisms across layers. Achieving this in practice, however, will require future research to more effectively navigate fundamental energy–performance trade-offs.

Paper roadmap. The rest of the paper is organized as follows. §2 presents our characterization methodology, including the definition of execution-idle and the measurement scope. §3 provides initial observations, and §4 reports the main findings on its prevalence and energy cost, especially for serving workloads. §5 discusses implications, §6 outlines broader system support for future GPU system design, and §7 reviews related work.

2 Datasets & Characterization Methodology

This section describes the dataset and methodology we use to characterize execution-idle in GPU workloads. We first describe the measurement environment and passive telemetry pipeline (§2.1), then define a taxonomy of GPU states (§2.2) and clarify the scope and limits of what these measurements can reveal (§2.3).

2.1 Measurement Setting and Data Collection

Cluster and study window. Our study uses passive telemetry from a large academic AI cluster running diverse GPU workloads, including training, batch inference, and online serving. Jobs are managed by Slurm [17] and receive exclusive whole-GPU allocations, without hardware partitioning such as MIG [29]. All measurements are collected under the cluster’s standard production configuration [15]: nodes use performance-oriented BIOS profiles, and GPUs operate under vendor-managed DVFS without fixed application clocks or additional power caps. Hence, our data characterizes how execution-idle appears under the default power behavior, rather than any manually tuned or customized settings.²

Our measurement includes 756 NVIDIA GPUs spanning multiple generations: 200 A6000 [36], 52 RTX 6000 Ada [35], 408 L40(S) [33], 64 A100 [30], 24 H100 [32], and 8 B200 [28]. Our dataset spans 31 days, from February 4, 2026 to March 7, 2026, and contains 162 GB of telemetry. Additional cluster details are listed in supplementary material.

Passive telemetry pipeline. We collect per-second GPU-, host-, and job-level telemetry from NVML [34], DCGM [31], OS counters, psutil [44], nvidia-smi [37], and Slurm [17]. These signals cover GPU power, activity, clocks, communication, host activity, timing, and job metadata; Table 1 summarizes representative fields.

Telemetry is collected passively on each compute node and then aligned with scheduler records using timestamps and GPU allocation metadata, allowing each GPU-second sample

²Recent Blackwell systems introduce data-center power profiles such as Max-Q, which improve efficiency through coarse-grained control across workload classes [24, 40]. These mechanisms are orthogonal to our study, since they do not perform fine-grained temporal adjustments to loaded-but-inactive intervals within executions.

Table 1. Representative signals collected by our passive profiling pipeline. These signals cover GPU power, activity, clocks, communication, host activity, timing, and job metadata.

Domain	Metric	Unit	Source	Description
GPU identity	hostname, gpu_id, gpu_name	–	Slurm, NVML	Identifies the host, GPU instance, and GPU model for each sampled record.
GPU power	power	W	NVML	GPU board power used for energy accounting.
GPU activity	sm, tensor, dram, fp16, fp32, fp64	%	DCGM	Compute and memory activity signals used to characterize whether the GPU is actively doing work. Some counters are unavailable on some GPU types.
GPU clocks	sm_clk, mem_clk	MHz	NVML	Runtime GPU clock frequencies used to study frequency behavior under different execution states.
GPU communication	pcie_tx, pcie_rx, nvlink_tx, nvlink_rx	MB/s	NVML, nvidia-smi	Device-side communication signals used to track PCIe and NVLink traffic. NVLink counters are unavailable on some GPU types.
Host activity	cpu_util, host_mem_util	%	psutil	Host-side CPU and memory utilization used to characterize surrounding system activity.
Network activity	nic_tx, nic_rx	MB/s	OS counters	Per-interface network throughput used to capture external data movement and network activity.
Timing	timestamp	s	Profiler	Per-sample timestamp used to align telemetry across sources and over time.
Job metadata	job_id, job_name	–	Slurm	Scheduler metadata used to associate telemetry with jobs and support job-level analysis.

to be attributed to a job. Each retained sample therefore represents one second of behavior on one allocated GPU for one job. We discard malformed records and samples that cannot be attributed reliably.

Profiling overhead. The profiling pipeline adds negligible overhead to compute nodes. According to `pidstat` [11], the profiling process remains below 0.05% CPU and uses roughly 400 MB of memory, about 0.08% of the nodes with more than 500 GB of host memory. Compressed telemetry logs require only 20–100 MB per server per day, negligible relative to the more than 5 TB of local storage available on each server.

User Privacy and Ethics. Under our institution’s policies, this research is considered exempt from review as our research does not concern human subjects. Nonetheless, we are sensitive to privacy concerns from cluster users. Hence, we spoke with PIs and cluster stakeholders to inform them of our study; we also supported an ‘opt-out’ flag for users to prevent data about their jobs from being captured. Finally, the dataset is anonymized and stripped of sensitive information that could link it to any particular researcher.

2.2 Defining Execution-Idle, Active, & Deep Idle States

Definitions The *execution-idle state* consists of intervals during job execution in which the GPU remains allocated and the program remains resident, yet visible activity is near zero. We classify an interval as execution-idle when all available compute- and memory-related signals—including

SM, tensor-core, other per-precision accelerator activity (e.g., fp32) when available, and DRAM activity—remain below 5%, and all available communication signals—including PCIe and NVLink traffic when available—remain below 1 GB/s ($\approx 3\%$ of PCIe 4.0 $\times$ 16 bandwidth). These conditions must hold simultaneously. If a signal is unavailable on a given GPU type, we omit it from the rule rather than treating it as violated.

Alongside *execution-idle*, we distinguish two other GPU states: *deep idle*, in which no program is resident and the GPU remains at baseline power, and *active execution*, in which a program is resident and observed activity exceeds the execution-idle threshold. Thus, *execution-idle* and *active execution* are both in-program states, whereas *deep idle* is not. The three states are mutually exclusive and collectively exhaustive. The key distinction is that, unlike *deep idle*, *execution-idle* occurs within a live job: a program remains resident, yet visible activity is low, and power can stay substantially above baseline despite little or no useful work.

We quantify time by summing samples assigned to each state. We quantify energy using NVML-reported board power: **integrating power over all samples yields total GPU energy**, while integrating only over execution-idle samples yields execution-idle energy. The ratio of these two quantities gives the fraction of GPU energy spent in execution-idle.

Conservative quantification of execution-idle intervals. Modern GPUs use dynamic voltage and frequency scaling (DVFS) [23] to adjust operating frequency in response to workload conditions, which in turn affects both device power and performance. Prior work suggests that modern GPUs

may take roughly 1–500 ms to adjust frequency [52]. Very short pauses may therefore be too brief for frequency to fall, recover, and manifest as a distinct operating regime.

To avoid counting such transient gaps as execution-idle when quantifying the cluster workloads, we adopt a conservative duration threshold of 5 s: an interval is counted as execution-idle only if it satisfies the low-activity conditions continuously for at least 5 s. This threshold is long enough to exclude brief pauses that existing DVFS mechanisms are intended to absorb, while still capturing sustained low-activity intervals that are long enough to matter for measurement and, potentially, system response. As a result, our methodology likely *underestimates*, rather than overstates, how often execution-idle occurs, and our main conclusions are not sensitive to this threshold (§4.3).

2.3 Scope and Limitations

Primary AI research cluster analysis. Our dataset comes from an academic AI cluster rather than a specialized production deployment, so its aggregate utilization is not meant to represent all industry GPU fleets. Compared with many production settings, academic clusters expose a broader mix of workload classes and user behaviors, including research workloads, short jobs, debugging runs, and interactive sessions. We therefore focus the later analyses on long-running, non-debug jobs, which better capture sustained training, batch inference, and serving behavior and more closely reflect where GPU energy is spent. This breadth is useful for characterization, since it reveals how execution-idle varies across workload classes within one environment. At the same time, many production GPU deployments are more specialized—for example, around training [57] or latency-sensitive serving—so bottlenecks that are diluted in a mixed academic trace may be even sharper in those settings. We examine that next with serving-oriented workloads.

For broad workload-level comparisons, we group academic jobs using keyword-based rules over job metadata; yielding the following groups of jobs: academic serving, academic batch inference, academic training, and academic others. These labels are intended only for coarse grouping rather than precise semantic classification.

Complementary industry-style replay. To corroborate that we might expect to see execution-idle states outside of academic workloads, we also study industry-style GPU usage using open-source serving systems driven by public traces derived from OpenAI [54], Qwen [53], and Azure [49]. These traces provide request arrival times together with input and output token lengths. Following state-of-the-art trace-driven serving evaluations [49], we synthesize requests from the traced token lengths and serve an open-source Llama-13B model [51] on L40S GPUs using vLLM [18]. Because the original traces were collected from deployments with larger fixed GPU pools than our testbed, we downscale each trace to

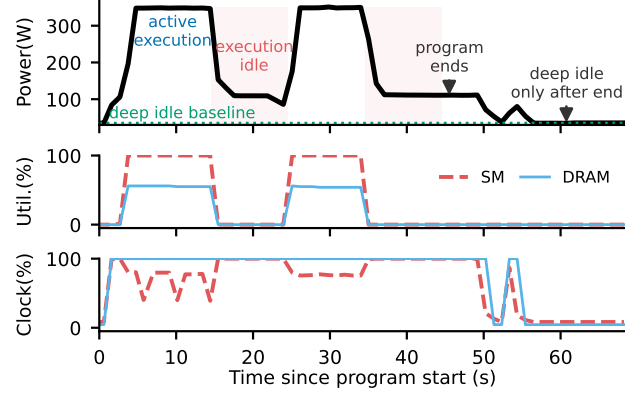
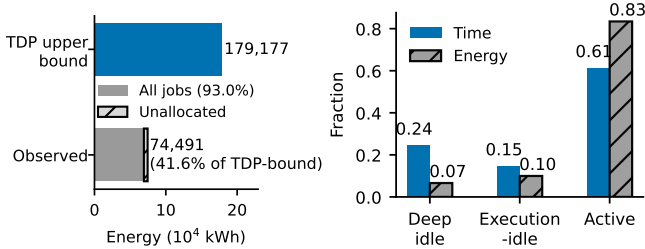


Figure 2. Time-aligned power, SM and DRAM utilization, and normalized frequency for a job on an L40S GPU, illustrating the execution-idle state.

a smaller but still fixed pool while preserving burstiness, and replay the resulting per-GPU streams for 30 minutes. This setup preserves the fixed-provisioning assumption of the original traced deployments (e.g., 32 GPUs in Qwen [53] and 96 GPUs in Azure [49]); autoscaling [10, 45, 61] is therefore outside the scope of our replay study. Because the replay uses one model, one GPU type, and one serving engine, it is not intended to fully characterize the magnitude of execution-idle across serving deployments. Instead, it serves as a complementary case study to test whether execution-idle also arises under more production-like serving demand.

Temporal and observability limits. Our passive measurements have two important limits. First, telemetry is sampled at 1 Hz, so very short bursts and sub-second transitions may be smoothed or missed. We therefore focus on sustained behavior rather than microsecond- or millisecond-scale dynamics. This is a deliberate trade-off: prior work [48] uses finer-grained logging to study short kernels and micro-kernels, whereas our goal is to characterize long-running deployed jobs, where sustained power draw accumulates into meaningful energy cost. At the scale of 756 GPUs over 31 days, 1 Hz sampling provides a practical and scalable resolution while preserving the temporal structure needed for cluster-scale energy accounting.

Second, current public vendor telemetry provides only limited component-level power visibility. NVIDIA interfaces expose device-level GPU power, and some newer datacenter platforms also report module or memory-subsystem power [37], but they do not expose a portable breakdown across compute, memory, and interconnect components. Because such counters are not available consistently across GPU generations in our cluster, we can quantify execution-idle and its device-level energy cost, but cannot attribute that cost uniformly to specific hardware components.



(a) Unallocated/job-attributed energy vs. TDP upper bound **(b)** Job-attributed GPU time and energy breakdown

Figure 3. Cluster-scale GPU energy accounting over the study window. The left panel compares observed GPU energy while the right panel decomposes job-attributed GPU time and energy by regime.

3 A First Look at Execution-Idle

Having defined our methodology, we now present an overview of the prevalence of execution-idle in our AI cluster, how the execution-idle state occurs, the magnitude of execution-idle power use relative to deep idle power use, hypothesize as to why execution-idle states occur, and demonstrate that execution-idle states exist across six generations of GPUs. In §4, we turn to quantifying the overall energy cost of execution-idle states within particular classes of workload.

Prevalence of execution-idle across the cluster. To focus on long-running workloads rather than debug or interactive sessions, we restrict attention to jobs lasting at least two hours (§4.3 evaluates sensitivity to this threshold). Even after a job is allocated to GPUs, those GPUs may not be used immediately: some jobs begin with CPU- or I/O-heavy setup, such as downloading a large model, leaving the GPU in deep idle. As a result, job-attributed GPU time and energy still fall into three states: deep idle, execution-idle, and active execution.

Figure 3b shows that deep idle accounts for 24% of job-attributed GPU time but only 7% of energy, consistent with the low power draw of hardware with no loaded program. Execution-idle, by contrast, accounts for 15% of time and 10% of energy. So while execution-idle occupies less time than deep idle, it consumes more energy because the program remains loaded and the GPU stays at elevated power.³ Active execution accounts for the remaining 61% of time and 83% of energy, and therefore dominates in-job GPU energy use.

How execution-idle appears during a job. Figure 2 illustrates all three GPU states in a measured academic job on an L40S GPU. During *active execution*, power rises to the expected high-power regime. During the highlighted

³For one cluster at our university alone, execution-idle states cost an estimated \$944 (at a US average rate of 13.6¢ per kWh), and produced approximately 2.58 – 2.80 metric tons of CO₂e (at a US average rate of 0.82-0.89 lbs of CO₂e per kWh) in a single month.

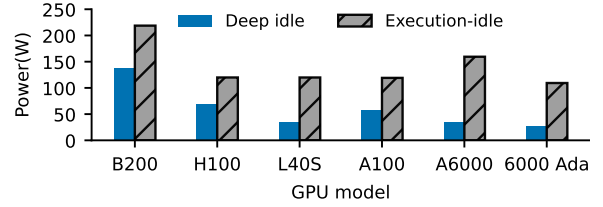


Figure 4. Power in the execution-idle state remains substantially above deep idle across all GPU models in our study.

execution-idle intervals, which last about 10 s, the job remains resident while visible compute, memory, and PCIe activity all drop to near zero, yet power stays around 110 W. Only after the program terminates does the GPU enter *deep idle*, where power drops to the baseline level of roughly 35 W. This example shows why execution-idle is a distinct operating state: it occurs within a live job, satisfies our low-activity definition, and still draws far more power than true idle.

Why execution-idle power remains high. As shown in Figure 3a, observed GPU energy use over the study is far below the fleet’s rated Thermal Design Power (TDP) upper bound. The cluster’s GPUs consumed 74,491 kWh in total, with 93% attributable to user jobs and the remaining 7% to unallocated or deep-idle periods. Overall, this is only 41.6% of the energy the same GPUs would have consumed if they had run continuously at TDP over the same interval. This difference reflects the fact that GPUs spend substantial time outside sustained active execution.

Execution-idle highlights why being below TDP is not the same as being energy-efficient. During execution-idle, visible activity drops to near zero, yet *clock frequencies often remain high*, so power stays far above deep idle even though little useful work is being performed.

We hypothesize that this behavior occurs due to an inability to guarantee fast response latency under truly deep idle states, and a choice to optimize for latency over energy efficiency. Power managers are tuned to ride through brief within-execution stalls by keeping clocks elevated, preserving responsiveness when work resumes quickly. That choice becomes costly when low-activity intervals persist (e.g., 10 s). In our cluster, across 4,596,723 execution-idle intervals, the median execution-idle duration is 9 s and the 90th percentile is 44 s. At those timescales, elevated frequency is no longer merely a responsiveness mechanism; it becomes a sustained source of energy cost. Accordingly, execution-idle accounts for 10.7% of runtime energy across 11,791 long-running jobs in the academic dataset.

We further test whether GPU power and frequency eventually downscale during prolonged execution-idle. Using a controlled experiment that extends execution-idle from 4 s to 2048 s, we find that both remain elevated even after 2048 s. Prolonged loaded-but-inactive intervals therefore remain power-disproportionate under default GPU behavior.

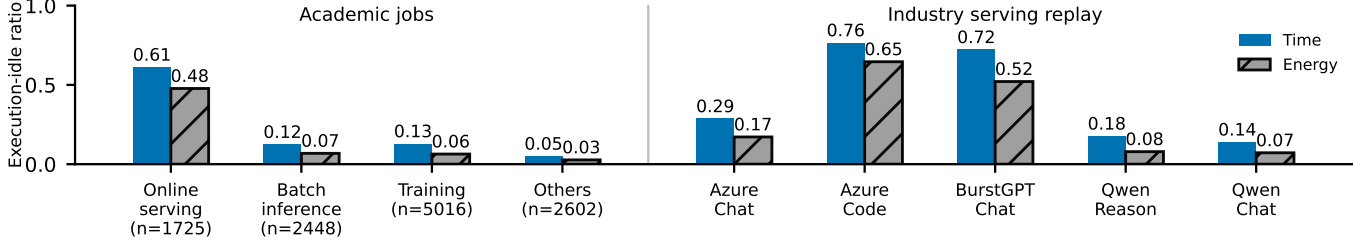


Figure 5. Execution-idle time and energy fractions across academic workload categories and replayed industry serving traces.

Execution-idle across GPU generations. Figure 4 shows that execution-idle appears across all GPU types in our study. Across every GPU model we measure, execution-idle draws substantially more power than deep idle, although the size of this gap varies across architectures and generations. This variation likely reflects hardware differences, but is difficult to attribute more precisely because comparable power breakdowns are not uniformly available across GPU generations. Still, the qualitative pattern is consistent: execution-idle appears across modern GPUs, including state-of-the-art NVIDIA B200 [28] GPUs.

4 Quantifying Execution-Idle Energy Waste

Using the dataset and execution-idle definition from §2, we answer five questions:

- How does execution-idle vary across workload categories, including academic jobs and replayed industry serving traces? (§4.1)
- How much time and energy does execution-idle consume across jobs? (§4.2)
- How sensitive is execution-idle to job lengths and inactivity requirements (§4.3)?
- How long do execution-idle intervals last? (§4.4)
- What likely causes tend to precede execution-idle? (§4.5)

Together, these analyses show that execution-idle is a meaningful but uneven component of GPU energy use. It accounts for a nontrivial share of cluster-wide GPU energy, is especially pronounced in serving workloads, is heavy-tailed across jobs, often persists long enough to matter, and follows recurring transfer, communication, and post-compute phases.

In-execution fractions. Before presenting the results in this section, we clarify the denominator used for all reported fractions. We are interested in the cost of execution-idle relative to the time and energy spent when a program is running on the GPU. For that reason, we exclude all deep-idle time and energy (where no program is active) from the denominator, including both (1) periods when no job is allocated to the GPU and (2) deep-idle periods within allocated jobs, such as during CPU- or I/O-heavy setup before the GPU is actually used.

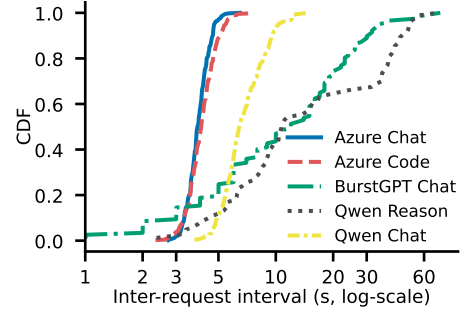


Figure 6. CDF of per-GPU inter-request intervals for replayed industry serving traces.

The denominator therefore consists only of *execution-idle* and *active execution*. This lets us ask: once a program is on the GPU, what fraction of execution time and energy is spent idle but still drawing elevated power? We call these *in-execution* fractions.

4.1 Variability of Execution-Idle Across Workloads

Execution-idle states appear in all workload categories. Using the workload labels described in §2, we group academic jobs into coarse categories and compare their execution-idle time and energy fractions, alongside the same metrics for replayed industry serving traces. Figure 5 shows that execution-idle appears in every category we examine, indicating that it is not confined to a single workload class.

Execution-idle cost is highest in serving workloads. Although execution-idle occurs in all jobs we observed, its magnitude varies sharply across workload categories. Among academic jobs, online serving (i.e., handling live latency-sensitive requests) is by far the most exposed regime: GPUs spend 61% of in-execution time in execution-idle, and 48% of energy is consumed during those intervals. Batch inference (i.e., offline processing over fixed inputs) and training spend much less energy and time in the execution-idle state, at 12–13% of time and 6–7% of energy, while the remaining workloads are lower still, at 5% of time and 3% of energy. Execution-idle is therefore broadly present, but its cost is highly uneven across workload classes.

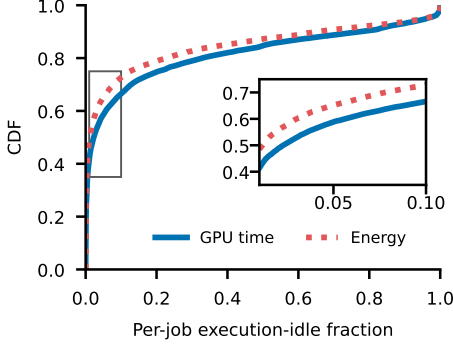


Figure 7. CDF of per-job execution-idle time and energy fractions.

Serving is especially exposed because GPUs remain resident under bursty demand. Serving keeps model state loaded on the GPU to preserve responsiveness, but request arrivals are uneven over time. As a result, GPUs often remain allocated and ready while little or no request work is actively executing, creating long loaded-but-low-activity intervals that still draw substantial power. Training and batch inference also exhibit execution-idle, but less persistently. In §4.5, we explore system conditions that correlate with execution idle-states and find that I/O use often precedes execution-idle, suggestive of applications blocking on I/O.

Industry serving workloads spent between 14-76% of time and 7-65% of energy in execution-idle states. To test whether the serving-side effect observed in our academic cluster also appears under industry-style demand, we replay several public serving traces using the method described in §2. Unlike the cluster study, where we impose a conservative 5 s minimum to avoid counting brief transients in opaque production jobs, the replay setting exposes the request arrival process directly. We therefore analyze all inter-request low-activity gaps in replay, rather than only those lasting at least 5 s. Figure 6 shows that per-GPU request streams remain bursty, with the time between consecutive requests often lasting several seconds. Median inter-request intervals are roughly 4–8 s across traces, while BurstGPT Chat and Qwen Reason exhibit heavier tails that extend well beyond 10 s. These results show that realistic serving traces naturally create frequent loaded-but-low-activity gaps.

Under this replay-specific accounting, replaying the industry serving trace on an L40S GPU, the results in Figure 5 mirror the qualitative pattern seen in the cluster. Low-activity periods account for 29%/17% of time/energy for Azure Chat, 76%/65% for Azure Code, 72%/52% for BurstGPT Chat, 18%/8% for Qwen Reason, and 14%/7% for Qwen Chat. Execution-idle is therefore not unique to the serving jobs observed in our cluster; it also appears under replayed industry demand traces. Its magnitude varies across traces, and more broadly will depend on the model, serving system, and GPU platform.

Both request spacing and duration shape execution-idle. Differences across traces reflect both how long GPUs wait between requests and how long each request occupies the GPU once admitted. Reasoning-heavy requests such as Qwen Reason keep the GPU busy longer, which reduces the fraction of time spent in execution-idle despite relatively long inter-request gaps. By contrast, shorter requests such as Azure Chat, and especially Azure Code, return the GPU to a loaded-but-inactive state more quickly, making second-scale gaps more costly.

Taken together, these results show that execution-idle is a general GPU phenomenon, but one whose cost is especially concentrated in serving. The replayed traces also point to a concrete driver: bursty demand acting on GPUs that remain resident and ready between requests.

4.2 Distribution of Execution Idle States Per Job

The distribution of job time spent in execution-idle is highly right-skewed, with 15.4% of jobs spending more than half of their time in execution-idle. We next examine how execution-idle is distributed across individual jobs. Figure 7 shows the CDF of per-job execution-idle fractions in both time and energy. A substantial tail of jobs spends a large share of time in this state: 33.4% of jobs spend more than 10% of time in execution-idle, 25.2% spend more than 20%, and 15.4% spend more than half of their time in execution-idle.

Some of this tail likely reflects serving-like burstiness, but it is not confined to serving. Among 11,791 long-running jobs, only 1,725 are confirmed serving jobs, or 14.6% of the population. Therefore, even under the extreme assumption that every serving job falls into the >20% execution-idle tail, at least 10.6% of all jobs in that tail must come from other workload categories. Execution-idle tail behavior therefore extends beyond serving and affects a broader set of workloads.

Highly right-skewed tail also appears in energy distribution. Overall, 27.1% of jobs spend more than 10% of energy in execution-idle, 21.2% spend more than 20%, and 12.8% spend more than half of energy in this state. Thus, execution-idle is not merely common on average; it is a severe energy cost for a nontrivial tail of jobs.

4.3 Effects of Inactivity Requirements and Job Length on Execution-Idle.

We next test whether our main estimates depend strongly on two conservative analysis choices: the long-job cutoff used to focus on sustained workloads and the minimum interval duration used to separate sustained execution-idle from brief transients.

Sensitivity to sustained-duration threshold. Our baseline requires low-activity intervals to persist for at least 5 s before counting them as execution-idle. Under 1 Hz passive telemetry, this is a conservative choice: a 1 s threshold is

Table 2. Execution-idle estimates during in-job execution under alternative thresholds.

Setting	Job cutoff	Min interval	Exec-idle time	Exec-idle energy
Baseline	≥ 2 h	5 s	19.17%	10.67%
Permissive interval	≥ 2 h	1 s	23.77%	13.91%
Conservative interval	≥ 2 h	10 s	15.6%	7.95%
Broader job set	≥ 1 h	5 s	19.22%	10.71%

more permissive and may include brief transients, whereas a 10 s threshold is stricter and counts only clearly sustained intervals. As expected, the measured magnitude changes with this threshold. Execution-idle accounts for 23.77% of in-execution time and 13.91% of energy with a 1 s threshold, compared with 15.6% of time and 7.95% of energy with a 10 s threshold. However, the qualitative conclusion is stable across all three settings. Even under the stricter 10 s definition, execution-idle remains substantial, showing that the phenomenon is not driven by threshold-edge events near the 5 s cutoff.

Sensitivity to job length. Our primary job-level analyses focus on jobs lasting at least 2 hours, which better capture sustained workloads while excluding short debug and interactive sessions. This scope also matches where most energy is consumed: over the study window, jobs running at least 2 hours account for 91% of all job-attributed GPU energy. To test sensitivity to this filter, we repeat the analysis with a 1-hour cutoff. As shown in Table 2, the resulting execution-idle estimates change negligibly, from 19.17% to 19.22% of in-execution time and from 10.67% to 10.71% of energy, indicating that our conclusions are not driven by the exact long-job threshold.

4.4 Duration of Individual Execution-Idle Periods

Over the study window, we identify 4,596,723 execution-idle intervals. Duration of each interval matters because longer intervals keep the GPU in a loaded but low-progress state for longer, allowing elevated power draw to accumulate into more energy overhead.

Execution-idle periods can last as long as minutes. We conservatively require an execution-idle interval to last at least 5 s before counting it, which already excludes short-lived fluctuations. In our academic-cluster measurement, even under this cutoff, Figure 8 shows a median interval of 9 s, while the 90th and 99th percentiles reach 44 s and 836 s, respectively.

How much energy saving is necessary to justify a latency penalty? We will discuss in §5 – and it is well-known in the literature [52] – that many energy savings techniques (especially frequency scaling) come with a cost in response time. Shifting from a low-frequency state to a high-frequency

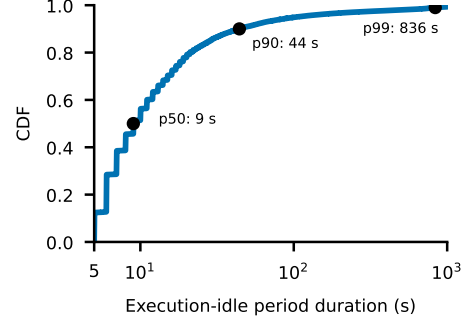


Figure 8. CDF of execution-idle interval durations.

state takes some time, and hence a request arriving while the GPU is in a low energy state will suffer increased service time. Studies report that GPUs take roughly 1–500 ms to adjust frequency [52]. Hence, engineers wish to avoid dropping the GPU to a low-frequency state only to immediately return to a high-frequency state: the energy savings do not justify the penalty in latency.

Reflecting on the data we observe, we philosophically wonder at what point the energy savings *do* justify a latency penalty for the next request. Is 44 s (0.00267 kWh on a B200 GPU) enough? 836 s (0.02783 kWh on a L40S GPU)?

4.5 What other System Metrics Correlate with Execution-Idle States?

With the exception of the industry serving tests, we do not have the ability to introspect into the running code in our cluster. As a result, it is not possible for us to causally identify what specific program behaviors are most common factors leading to execution-idle times. Nonetheless, we can identify probable candidates by studying what system-level conditions tend to precede execution-idle. For each execution-idle interval, we extract up to 10 s of preceding device- and host-side telemetry, truncating the window when necessary so that it contains only the nearest preceding active-execution segment. We then apply HDBSCAN [22] to group these pre-idle windows into recurring patterns, and manually analyze the salient clusters through their telemetry signatures to assign likely causes.

A small number of factors dominate the windows preceding execution-idle. Figure 9 shows that most execution-idle onsets are associated with just a few categories. PCIe-heavy intervals account for the largest share at 48%, followed by compute-to-idle at 33%, NIC-heavy at 17%, and NVLink-heavy at 2%. This distribution is shaped in part by our cluster’s hardware mix. Several deployed GPU models, including L40S and 6000 Ada, do not support NVLink and therefore depend more on NIC-based communication. Accordingly, NVLink-heavy intervals are concentrated on higher-end data-center GPUs in our cluster, such as A100.

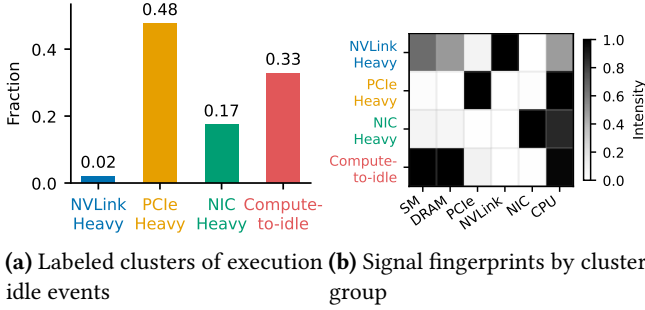


Figure 9. Signals from the interval immediately preceding execution-idle, grouped into inferred categories.

As shown in Figure 9b, the fingerprints are consistent with distinct system-level behaviors. The PCIe-heavy category exhibits elevated PCIe and CPU activity, consistent with host-device transfer or coordination overhead, and is likely common in data loading, preprocessing, and framework-managed execution pipelines. The NIC-heavy category shows elevated NIC and CPU activity, suggesting distributed communication or storage-related movement such as NFS traffic, and thus aligns naturally with multi-node training and communication-intensive jobs. The NVLink-heavy category exhibits strong NVLink activity and is consistent with intra-node GPU-GPU communication in multi-GPU training on NVLink-connected servers.

Finally, the compute-to-idle category shows elevated SM and DRAM activity immediately before idle onset, followed by a transition into near-zero activity during execution-idle. This pattern is consistent with workloads that alternate between bursts of GPU work and waiting, including bursty serving as well as training pipelines that pause at synchronization or coordination boundaries.

4.6 Summary

Across cluster-scale accounting, workload breakdowns, per-job distributions, interval durations, and pre-idle signatures, a consistent picture emerges. Execution-idle is neither a corner case nor a purely serving-specific artifact. It accounts for a non-trivial share of GPU energy, appears across workload classes, becomes severe for a substantial tail of jobs, often lasts long enough to matter, and tends to arise after recognizable transfer, communication, or post-compute phases. At the same time, its cost is highly uneven, with serving workloads standing out as the most exposed regime.

5 Implications

The existence of a state in which a GPU is doing nearly zero useful work, but nonetheless consuming power intuitively demands a fix. While we will not be able to solve the problem altogether in the rest of paper, we briefly discuss three implications for developers of AI software.

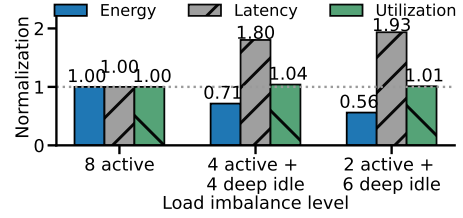


Figure 10. Energy, p95 latency, average GPU utilization at different load imbalance level, normalized to 8-active-GPU baseline.

- **Overall cluster utilization should not be used as a proxy metric for power draw.**(§5.1)
- **Ongoing research to improve utilization within a GPU are likely to improve energy efficiency.**(§5.2)
- **Simple, manual overrides to frequency scaling can, predictably, save energy at some latency cost.** (§5.3)

5.1 Energy Draw Can Vary Between Deployments at the Same Nominal Utilization

Operators and researchers often talk of cluster-level SM utilization figures as an implicit proxy for energy efficiency. Because deep-idle states also consume power, operators seek to avoid leaving GPUs underutilized where they are perceived to be wasting power that could instead be applied to meaningful work.

However, energy waste due to execution-idle states means that *between two deployments operating at the same level of utilization, power draw can differ* and that in fact, *it may be better from an energy perspective to leave some cluster GPUs unutilized in favor of packing more work onto the same GPU* (at least from an energy perspective).

We perform an experiment with a biased load balancer. Rather than spreading requests evenly across the entire pool, which leaves many GPUs lightly active and repeatedly exposed to short execution-idle intervals, the scheduler deliberately introduces *load imbalance*: concentrate work onto fewer GPUs while leaving the others in deep idle state.

Setup. We study this setting using an 8-GPU serving pool built by downsampling the original 96-GPU Azure Code traces [49]. We compare three cases: (1) a balanced baseline with all 8 GPUs active and no downscaling; (2) a 4-active-GPU case, where 4 GPUs carry the workload and the other 4 remain lightly loaded and downscaled; and (3) a 2-active-GPU case, where 2 GPUs carry the workload and the other 6 remain lightly loaded and downscaled.

Energy nearly halves, even though utilization stays almost the same. As Figure 10 shows for Azure Code, deliberately concentrating work onto fewer GPUs cuts total GPU energy to 56% of the balanced case, while overall SM

Algorithm 1 Execution-Idle-Aware Frequency Control

Require: threshold X , cooldown Y , clocks $f_{\max}, f_{\min}$

```

1:  $c \leftarrow 0, t_{\text{cooldown}} \leftarrow 0, \text{downscaled} \leftarrow \text{false}$ 
2: for each  $\epsilon$  second control interval at time  $t$  do
3:   Read  $\text{sm}, \text{tensor}, \text{fp16}, \text{dram}, \text{pcie}, \text{nvlink}, \dots$ 
4:    $a_{\text{comp}} \leftarrow \max(\text{sm}, \text{tensor}, \text{fp16}, \dots)$ 
5:    $a_{\text{mem}} \leftarrow \text{dram}$ 
6:    $a_{\text{comm}} \leftarrow \max(\text{pcie}, \text{nvlink})$ 
7:   if  $a_{\text{comp}} < 0.05$  and  $a_{\text{mem}} < 0.05$  and  $a_{\text{comm}} < 1 \text{ GB/s}$  then
8:      $c \leftarrow c + \epsilon$ 
9:   else
10:     $c \leftarrow 0$ 
11:    if  $\text{downscaled}$  then
12:      Set GPU clock to  $f_{\max}$ 
13:       $\text{downscaled} \leftarrow \text{false}$ 
14:       $t_{\text{cooldown}} \leftarrow t + Y$ 
15:    if  $c > X$  and  $t \geq t_{\text{cooldown}}$  and  $\neg \text{downscaled}$  then
16:      Set GPU clock to  $f_{\min}$ 
17:       $\text{downscaled} \leftarrow \text{true}$ 

```

utilization changes little. Looking only at utilization would therefore suggest similar power draw across the two configurations, which is misleading.

This happens because average utilization does not reflect how energy is distributed across the GPU pool. Under imbalance, the same total work is concentrated on fewer active GPUs, while the rest remain in deep idle. Pool-wide utilization therefore stays similar, yet total energy falls because fewer GPUs remain in higher-power serving states. The energy saved by keeping more GPUs in deep idle outweighs the extra energy drawn by the more heavily loaded GPUs. In this setting, utilization masks the crucial difference between work performed and the number of GPUs still consuming baseline and execution-idle power.

At the same time, imbalance introduces a latency penalty.

Looking at a reduced energy cost for the same level of cluster utilization, one might be tempted to run all clusters at a deliberate skew. As load is concentrated onto fewer GPUs, serving latency rises. With 4 active GPUs, p95 request latency increases by 80%; with 2 active GPUs, the increases grow by 93%, as shown in Figure 10. Hence, we do not prescribe highly skewed load balancing as a solution to execution idle: we merely offer it as a cautionary tale that utilization figures do not paint a complete picture of energy use.

5.2 Increasing Utilization Within a Single GPU Does Improve Efficiency.

Although cluster-level utilization metrics are misleading, as we discuss above, individual increased GPU utilization does result in improved energy efficiency. Because there is a fixed cost to keeping a GPU in an active and loaded state (as observed in Figure 1, it is best to keep those GPUs busy.

Following this intuition, we suspect that *co-serving* systems are likely to correlate with improved energy efficiency.

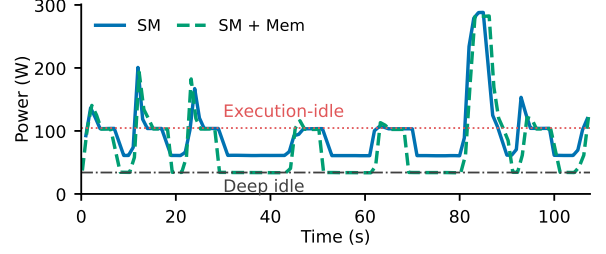


Figure 11. Power over time under SM-only and SM+memory execution-idle-aware frequency control.

These systems improve utilization by packing complementary workloads onto fewer devices, for example by co-serving online and offline jobs [42], serving multiple LLMs concurrently [60], or co-serving fine-tuning and inference [38].

On the other hand, *autoscaling* systems such as BlitzScale [61], ServerlessLLM [10], and INFaaS [45] have less clear energy implications. Their primary goal is elasticity and SLO preservation rather than direct energy minimization. By scaling in excess capacity and consolidating load, they may reduce execution-idle indirectly. However, aggressive scale-out to avoid latency degradation can also increase the number of active GPUs and thereby hurt energy efficiency. In this context, we see the need for direct power measurements as a key metric for system evaluation, as these systems largely report latency, throughput, utilization, or cloud cost.

5.3 Software-induced Frequency Control is a Useful Lever Today

The most direct response to execution-idle is to conservatively downscale frequency during low-progress intervals. We perform a simple experiment to override the default DVFS and instead institute our own, more aggressive algorithm in software. Similar power-management studies and systems, such as DynamoLLM [49] and μ -Serve [43] have used GPU frequency tuning and power capping as a configuration knob in their energy-performance optimization. Our goal here is narrower: we use a lightweight, local controller as a baseline to test whether execution-idle can be made less costly by reacting directly to sustained low-progress intervals. As shown in Algorithm 1, our controller waits for several consecutive seconds of near-zero activity before lowering the device to the minimum available frequency, restores the original setting when activity resumes, and then holds that setting for a short cooldown period to avoid rapid oscillation. In our implementation, the controller uses a 3 s trigger threshold and a 5 s cooldown period.

Setup. We replay the Azure Code serving trace for 1175 s on an L40S GPU under two frequency-control configurations while serving the same total number of requests. Using `nvidia-smi` [37], we lower either (1) the graphics/compute clock alone or (2) both the graphics/compute and memory

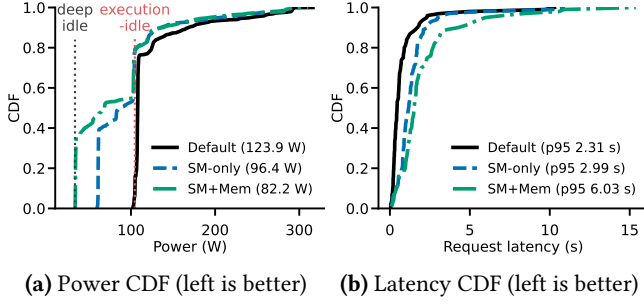


Figure 12. Power-latency trade-off of execution-idle-aware frequency downscaling.

clocks; finer-grained component-specific controls are not exposed. Because replay duration is fixed, we use average GPU power as a proxy for total energy.

Downscaling more aggressively reduces the cost of deep idle states. Figure 11 shows that online downscaling can react at the timescale of execution-idle intervals that commonly appear in our measurements (e.g., 10 s). On this platform, setting only the SM-related clock to the available minimum reduces execution-idle power from 105 W to 61 W, while lowering both SM and memory clocks further reduces it to 35 W (deep idle power).

Across the full replay, SM-only downscaling reduces average power from 123.9 W to 96.4 W, a 22% reduction. Lowering both SM and memory clocks reduces average power further to 82.2 W, a 34% reduction.

As expected, the algorithm does come with latency penalties. These savings, however, come with clear latency penalties. As shown in Figure 12, p95 latency rises from 2.31 s to 2.99 s (29%) under SM-only downscaling, and to 6.03 s (160%) when both SM and memory clocks are reduced aggressively.

The benefit of the approach is that it can be managed by the operator. Is a 160% increase in latency an acceptable cost in exchange for a 34% power reduction? Is a 29% increase in latency in exchange for a 22% reduction a better deal? Or is no latency cost acceptable?

From a hardware vendor’s perspective, GPUs typically compete on performance and so it is no surprise that default GPU configurations opt for better latency and higher energy. However, for operators and developers who may wish to strike a different bargain, the levers already exist to make a different choice (and likely, with application-aware context, an even better choice than the simple and naïve algorithm above).

6 What Broader System Support is Needed to Manage Execution-Idle in Future Systems?

Before concluding, we reflect on a few research directions that might improve energy waste due to execution-idle periods.

Workload-power interfaces for execution-idle management. Execution-idle should not be treated purely as a device-level phenomenon. Its frequency, duration, and performance sensitivity depend strongly on workload structure: some jobs can tolerate aggressive downscaling during low-progress periods, while others cannot. This suggests a workload-power co-design opportunity, where applications, system software, or serving frameworks expose signals such as burstiness, slack, communication phases, or latency sensitivity to lower layers. With this information, the system could better decide when execution-idle is safe to exploit, when it lies on the critical path, and how aggressively to trade performance for energy. More generally, systems should expose the power implications of execution-idle explicitly, rather than forcing hardware control to infer them indirectly from utilization counters alone.

SLO-aware execution-idle control for latency-sensitive serving. For latency-sensitive services, execution-idle-aware control should be integrated into SLO-driven resource management rather than applied in isolation. Prior work such as μ -Serve [43] and DynamoLLM [49] shows that dynamic GPU-frequency control can reduce serving energy while meeting latency SLOs. Our results suggest that execution-idle provides a complementary signal for such policies: even under SLO-aware control, it can still arise as a distinct low-progress regime within execution, where deeper temporary downscaling may be worthwhile. An important open question is how to combine execution-idle detection with queueing state, burst forecasts, slack, and tail-latency objectives in a unified serving controller.

From device-level to component-aware power proportionality. Our study focuses on device-level power proportionality: execution-idle keeps whole-device GPU power elevated despite near-zero visible activity, and current controls can reduce part of that cost. At the same time, our results suggest that this inefficiency is not tied to a single component: lowering SM frequency reduces execution-idle power substantially, and lowering memory frequency reduces it further. A natural next step is component-aware power proportionality. Future systems could ask whether execution-idle also exists within individual subsystems, such as compute, memory, and communication, and whether components off the critical path can be downscaled independently. Realizing this would require richer component-level observability and more flexible control across GPU subsystems.

In summary, these directions reinforce the same message: execution-idle should be treated as an important power state. It is a recurring and costly operating regime, and future systems should detect it, reason about it, and manage it explicitly.

7 Related Work

Energy proportionality has been studied extensively in CPU-centric systems. Classic work [1, 8] on energy-proportional computing asks how closely server or CPU power tracks utilization. This question is now especially important for GPUs. As AI demand grows, GPUs account for an increasing share of data-center power [6, 14, 47], making GPU energy efficiency increasingly critical. As a result, GPU power proportionality is no longer merely a device-level concern, but an important determinant of overall data-center energy efficiency.

Prior GPU energy work focuses mainly on aggregate efficiency and operating points. A broad body of important work takes an end-to-end perspective on GPU energy, studying metrics such as energy per token or energy to solution [20], and evaluating how efficiency varies with frequency [5, 50, 62], batch size [9, 25, 58], request shape [9, 55], model choice [4, 21, 25], serving engine [26], hardware platform [3, 4], and request load [16, 59]. This literature has been invaluable in establishing that GPU energy efficiency is highly workload- and configuration-dependent.

Recent measurement studies characterize GPU power and utilization across phases and deployments. Recent work measures power-management opportunities for LLM inference in the cloud and characterizes GPU utilization patterns in large-scale systems [16, 19, 25, 39, 46, 48]. These studies show substantial variation in GPU power and activity across jobs, phases, and deployment settings, and they provide important foundations for reasoning about GPU energy beyond coarse aggregate metrics.

Runtime power-management and serving systems provide important control mechanisms. Related serving and control systems such as μ -Serve [43], DynamoLLM [49], BlitzScale [61], ServerlessLLM [10] and vendor mechanisms such as Blackwell datacenter power profiles [24, 40] show how energy can be improved through frequency control, autoscaling, and device-level reconfiguration. These mechanisms are closely related to our setting because they can change either the cost of low-activity periods or the system conditions under which such periods arise.

Taken together, this literature shows that understanding GPU energy requires looking beyond aggregate utilization to how power and activity co-evolve over time, and to where GPU energy is spent productively versus unproductively. Our work complements these efforts

by isolating *execution-idle* as a recurring loaded-but-low-activity regime, quantifying its prevalence and cost in real deployments, and showing why it warrants explicit attention in future GPU system design.

8 Conclusion

We identify *execution-idle* as a distinct GPU operating regime in which a program remains loaded, visible activity is near zero, yet power remains well above deep idle. Across a 31-day cluster study and replayed serving traces, we show that execution-idle is common, consumes a meaningful share of GPU energy, and is especially important for bursty serving workloads. We hypothesize that this cost arises because current GPU power behavior is tuned to preserve responsiveness across brief stalls by keeping clocks and power elevated; in real workloads, however, these low-activity periods often persist long enough for the energy cost to accumulate. We further show that execution-idle can be mitigated either by lowering its cost through downscaling or by reducing exposure to it through scheduling, although both approaches introduce explicit energy–performance trade-offs. Taken together, these findings argue that future GPU systems should treat execution-idle as a first-class operating state and manage it explicitly in the pursuit of more energy-efficient AI infrastructure.

References

- [1] Luiz André Barroso and Urs Hölzle. 2007. The Case for Energy-Proportional Computing. *Computer* 40, 12 (2007), 33–37. doi:10.1109/MC.2007.443
- [2] Sangjin Choi, Inho Koo, Jeongseob Ahn, Myeongjae Jeon, and Youngjin Kwon. 2023. EnvPipe: Performance-preserving DNN Training Framework for Saving Energy. In *2023 USENIX Annual Technical Conference (USENIX ATC 23)*. USENIX Association, Boston, MA, 851–864. <https://www.usenix.org/conference/atc23/presentation/choi>
- [3] Jae-Won Chung, Jeff J. Ma, Ruofan Wu, Jiachen Liu, Oh Jun Kweon, Yuxuan Xia, Zhiyu Wu, and Mosharaf Chowdhury. 2025. The ML.ENERGY Benchmark: Toward Automated Inference Energy Measurement and Optimization. arXiv:2505.06371 [cs.LG] <https://arxiv.org/abs/2505.06371>
- [4] Jae-Won Chung, Ruofan Wu, Jeff J. Ma, and Mosharaf Chowdhury. 2026. Where Do the Joules Go? Diagnosing Inference Energy Consumption. arXiv:2601.22076 [cs.LG] <https://arxiv.org/abs/2601.22076>
- [5] Mariana Toledo Costa, Antigoni Georgiadou, III White, James B., Bruno Villasenor Alvarez, Jordà Polo, Woong Shin, Philippe Olivier Alexandre Navaux, Bronson Messer, and Arthur Francisco Lorenzon. 2025. Characterizing the Impact of GPU Power Management on an Exascale System. In *Proceedings of the SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC Workshops '25)*. Association for Computing Machinery, New York, NY, USA, 1524–1533. doi:10.1145/3731599.3767702
- [6] Electric Power Research Institute. 2026. *Powering Intelligence: Analyzing Artificial Intelligence and Data Center Energy Consumption*. Technical Report 3002034696. Electric Power Research Institute (EPRI). <https://www.epri.com/research/products/000000003002034696>
- [7] Luke Emberson and Ben Cottier. 2025. GPUs Account for About 40% of Power Usage in AI Data Centers. <https://epoch.ai/data-insights/gpus-power-usage-in-ai-data-centers> Epoch AI analysis.

- [8] Xiaobo Fan, Wolf-Dietrich Weber, and Luiz Andre Barroso. 2007. Power provisioning for a warehouse-sized computer. *SIGARCH Comput. Archit. News* 35, 2 (June 2007), 13–23. doi:10.1145/1273440.1250665
- [9] Jared Fernandez, Clara Na, Vashisth Tiwari, Yonatan Bisk, Sasha Luccioni, and Emma Strubell. 2025. Energy Considerations of Large Language Model Inference and Efficiency Optimizations. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar (Eds.). Association for Computational Linguistics, Vienna, Austria, 32556–32569. doi:10.18653/v1/2025.acl-long.1563
- [10] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. 2024. ServerlessLLM: Low-Latency Serverless Inference for Large Language Models. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 135–153. <https://www.usenix.org/conference/osdi24/presentation/fu>
- [11] Sébastien Godard. 2025. sysstat: Performance Monitoring Tools for Linux. <https://github.com/sysstat/sysstat> Includes the pidstat utility; accessed 2026-03-30.
- [12] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
- [13] Alan Gray. 2024. Maximizing Energy and Power Efficiency in Applications with NVIDIA GPUs. <https://developer.nvidia.com/blog/maximizing-energy-and-power-efficiency-in-applications-with-nvidia-gpus/> NVIDIA Technical Blog.
- [14] Alastair Green, Humayun Tai, Jesse Noffsinger, Pankaj Sachdeva, Arijita Bhan, and Raman Sharma. 2024. How Data Centers and the Energy Sector Can Sate AI’s Hunger for Power. <https://www.mckinsey.com/industries/private-capital/our-insights/how-data-centers-and-the-energy-sector-can-sate-ais-hunger-for-power> McKinsey & Company article.
- [15] Hewlett Packard Enterprise. 2026. Workload profiles | HPE iLO 5 User Guide. HPC profile disables power management to optimize sustained bandwidth and compute capacity.
- [16] Ali Jahanshahi, Hadi Zamani Sabzi, Chester Lau, and Daniel Wong. 2020. GPU-NEST: Characterizing Energy Efficiency of Multi-GPU Inference Servers. *IEEE Computer Architecture Letters* 19, 2 (2020), 139–142. doi:10.1109/LCA.2020.3023723
- [17] M Jette, C Dunlap, J Garlick, and M Grondona. 2002. SLURM: Simple Linux Utility for Resource Management. (07 2002). <https://www.osti.gov/biblio/15002962>
- [18] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.
- [19] Imran Latif, Alex C. Newkirk, Matthew R. Carbone, Arslan Munir, Yuewei Lin, Jonathan Koomey, Xi Yu, and Zhihua Dong. 2025. Single-Node Power Demand During AI Training: Measurements on an 8-GPU NVIDIA H100 System. *IEEE Access* 13 (2025), 61740–61747. doi:10.1109/ACCESS.2025.3554728
- [20] Alexandra Sasha Luccioni, Sylvain Viguier, and Anne-Laure Ligozat. 2023. Estimating the carbon footprint of BLOOM, a 176B parameter language model. *J. Mach. Learn. Res.* 24, 1, Article 253 (Jan. 2023), 15 pages.
- [21] Sasha Luccioni, Yacine Jernite, and Emma Strubell. 2024. Power Hungry Processing: Watts Driving the Cost of AI Deployment?. In *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency (Rio de Janeiro, Brazil) (FAcT ’24)*. Association for Computing Machinery, New York, NY, USA, 85–99. doi:10.1145/3630106.3658542
- [22] Leland McInnes, John Healy, and Steve Astels. 2017. hdbscan: Hierarchical density based clustering. *Journal of Open Source Software* 2, 11 (2017), 205. doi:10.21105/joss.00205
- [23] Xinxin Mei, Qiang Wang, and Xiaowen Chu. 2016. A Survey and Measurement Study of GPU DVFS on Energy Conservation. arXiv:1610.01784 [cs.DC] <https://arxiv.org/abs/1610.01784>
- [24] Sreedhar Narayanaswamy, Pratikkumar Dilipkumar Patel, Ian Karlin, Apoorv Gupta, Sudhir Saripalli, and Janey Guo. 2025. Datacenter Energy Optimized Power Profiles. arXiv:2510.03872 [cs.DC] <https://arxiv.org/abs/2510.03872>
- [25] Chenxu Niu, Wei Zhang, Jie Li, Yongjian Zhao, Tongyang Wang, Xi Wang, and Yong Chen. 2025. TokenPowerBench: Benchmarking the Power Consumption of LLM Inference. arXiv:2512.03024 [cs.LG] <https://arxiv.org/abs/2512.03024>
- [26] Chenxu Niu, Wei Zhang, Yongjian Zhao, and Yong Chen. 2025. Energy Efficient or Exhaustive? Benchmarking Power Consumption of LLM Inference Engines. *SIGENERGY Energy Inform. Rev.* 5, 2 (Aug. 2025), 56–62. doi:10.1145/3757892.3757900
- [27] NVIDIA. 2025. Driver Persistence. <https://docs.nvidia.com/deploy/driver-persistence/index.html>
- [28] NVIDIA. 2025. NVIDIA Blackwell B200 GPU. <https://images.nvidia.com/aem-dam/Solutions/documents/HGX-B200-PCF-Summary.pdf>
- [29] NVIDIA Corporation. 2025. NVIDIA Multi-Instance GPU User Guide. <https://docs.nvidia.com/datacenter/tesla/mig-user-guide/index.html>
- [30] NVIDIA Corporation. 2026. NVIDIA A100 Tensor Core GPU. <https://www.nvidia.com/en-us/data-center/a100/>
- [31] NVIDIA Corporation. 2026. NVIDIA Data Center GPU Manager (DCGM) Documentation. <https://docs.nvidia.com/datacenter/dcgml/latest/index.html>
- [32] NVIDIA Corporation. 2026. NVIDIA H100 GPU. <https://www.nvidia.com/en-us/data-center/h100/>
- [33] NVIDIA Corporation. 2026. NVIDIA L40S GPU for AI and Graphics Performance. <https://www.nvidia.com/en-us/data-center/l40s/>
- [34] NVIDIA Corporation. 2026. NVIDIA Management Library (NVML) API Reference Guide. <https://docs.nvidia.com/deploy/nvml-api/index.html>
- [35] NVIDIA Corporation. 2026. NVIDIA RTX 6000 Ada Generation Graphics Card. <https://www.nvidia.com/en-us/products/workstations/rtx-6000/>
- [36] NVIDIA Corporation. 2026. NVIDIA RTX A6000. <https://www.nvidia.com/en-us/products/workstations/rtx-a6000/>
- [37] NVIDIA Corporation. 2026. NVIDIA System Management Interface (nvidia-smi). <https://docs.nvidia.com/deploy/nvidia-smi/>
- [38] Gabriele Oliaro, Xupeng Miao, Xinhao Cheng, Vineeth Kada, Mengdi Wu, Ruohan Gao, Yingyi Huang, Remi Delacourt, April Yang, Yingcheng Wang, Colin Unger, and Zhihao Jia. 2025. FlexLLM: Token-Level Co-Serving of LLM Inference and Finetuning with SLO Guarantees. arXiv:2402.18789 [cs.DC] <https://arxiv.org/abs/2402.18789>
- [39] Pratyush Patel, Esha Choukse, Chaojie Zhang, Íñigo Goiri, Brijesh Warrier, Nithish Mahalingam, and Ricardo Bianchini. 2024. Characterizing Power Management Opportunities for LLMs in the Cloud. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3 (La Jolla, CA, USA) (ASPLOS ’24)*. Association for Computing Machinery, New York, NY, USA, 207–222. doi:10.1145/3620666.3651329
- [40] Pratikkumar Patel and Sreedhar Narayanaswamy. 2025. Optimize Data Center Efficiency for AI and HPC Workloads with Power Profiles. <https://developer.nvidia.com/blog/optimize-data-center-efficiency-for-ai-and-hpc-workloads-with-power-profiles/>
- [41] David Patterson, Joseph Gonzalez, Urs Hölzle, Quoc Le, Chen Liang, Lluis-Miquel Munguia, Daniel Rothchild, David R. So, Maud Texier, and Jeff Dean. 2022. The Carbon Footprint of Machine Learning Training Will Plateau, Then Shrink. *Computer* 55, 7 (2022), 18–28. doi:10.1109/MC.2022.3148714

- [42] Yifan Qiao, Shu Anzai, Shan Yu, Haoran Ma, Shuo Yang, Yang Wang, Miryung Kim, Yongji Wu, Yang Zhou, Jiarong Xing, Joseph E. Gonzalez, Ion Stoica, and Harry Xu. 2025. ConServe: Fine-Grained GPU Harvesting for LLM Online and Offline Co-Serving. arXiv:2410.01228 [cs.DC] <https://arxiv.org/abs/2410.01228>
- [43] Haoran Qiu, Weichao Mao, Archit Patke, Shengkun Cui, Saurabh Jha, Chen Wang, Hubertus Franke, Zbigniew Kalbarczyk, Tamer Başar, and Ravishankar K. Iyer. 2024. Power-aware Deep Learning Model Serving with μ -Serve. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. USENIX Association, Santa Clara, CA, 75–93. <https://www.usenix.org/conference/atc24/presentation/qiu>
- [44] Giampaolo Rodolà. 2026. psutil: Cross-platform lib for process and system monitoring in Python. <https://github.com/giampaolo/psutil>
- [45] Francisco Romero, Qian Li, Neeraja J. Yadwadkar, and Christos Kozyrakis. 2021. INFaaS: Automated Model-less Inference Serving. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. USENIX Association, 397–411. <https://www.usenix.org/conference/atc21/presentation/romero>
- [46] Siddharth Samsi, Dan Zhao, Joseph McDonald, Baolin Li, Adam Michaleas, Michael Jones, William Bergeron, Jeremy Kepner, Devshv Tiwari, and Vijay Gadepally. 2023. From Words to Watts: Benchmarking the Energy Costs of Large Language Model Inference. arXiv:2310.03003 [cs.CL] <https://arxiv.org/abs/2310.03003>
- [47] Arman Shehabi, Sarah Josephine Smith, Alex Hubbard, Alexander Newkirk, Nuoa Lei, Md AbuBakar Siddik, Billie Holecek, Jonathan G. Koomey, Eric R. Masanet, and Dale A. Sartor. 2024. *2024 United States Data Center Energy Usage Report*. Technical Report. Lawrence Berkeley National Laboratory. doi:10.71468/P1WC7Q
- [48] Varsha Singhanian, Shaizeen Aga, and Mohamed Assem Ibrahim. 2025. FinGraV: Methodology for Fine-Grain GPU Power Visibility and Insights. arXiv:2412.12426 [cs.AR] <https://arxiv.org/abs/2412.12426>
- [49] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Josep Torrellas, and Esha Choukse. 2025. DynamoLLM: Designing LLM Inference Clusters for Performance and Energy Efficiency. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. 1348–1362. doi:10.1109/HPCA61900.2025.00102
- [50] Zhenheng Tang, Yuxin Wang, Qiang Wang, and Xiaowen Chu. 2019. The Impact of GPU DVFS on the Energy and Performance of Deep Learning: an Empirical Study. In *Proceedings of the Tenth ACM International Conference on Future Energy Systems* (Phoenix, AZ, USA) (*e-Energy '19*). Association for Computing Machinery, New York, NY, USA, 315–325. doi:10.1145/3307772.3328315
- [51] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971 [cs.CL] <https://arxiv.org/abs/2302.13971>
- [52] Daniel Velicka, Ondrej Vysocky, and Lubomir Riha. 2025. Methodology for GPU Frequency Switching Latency Measurement. arXiv:2502.20075 [cs.DC] <https://arxiv.org/abs/2502.20075>
- [53] Jiahao Wang, Jinbo Han, Xingda Wei, Sijie Shen, Dingyan Zhang, Chenguang Fang, Rong Chen, Wenyuan Yu, and Haibo Chen. 2025. KVCache Cache in the Wild: Characterizing and Optimizing KVCache Cache at a Large Cloud Provider. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. USENIX Association. <https://www.usenix.org/conference/atc25/presentation/wang-jiahao>
- [54] Yuxin Wang, Yuhan Chen, Zeyu Li, Xueze Kang, Yuchun Fang, Yeju Zhou, Yang Zheng, Zhenheng Tang, Xin He, Rui Guo, Xin Wang, Qiang Wang, Amelie Chi Zhou, and Xiaowen Chu. 2025. BurstGPT: A Real-World Workload Dataset to Optimize LLM Serving Systems. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2 (KDD '25)*. ACM, Toronto, ON, Canada. doi:10.1145/3711896.3737413
- [55] Grant Wilkins, Srinivasan Keshav, and Richard Mortier. 2024. Offline Energy-Optimal LLM Serving: Workload-Based Energy Models for LLM Inference on Heterogeneous Systems. arXiv:2407.04014 [cs.DC] <https://arxiv.org/abs/2407.04014>
- [56] Carole-Jean Wu, Ramya Raghavendra, Udit Gupta, Bilge Acun, Newsha Ardalani, Kiwan Maeng, Gloria Chang, Fiona Aga Behram, James Huang, Charles Bai, Michael Gschwind, Anurag Gupta, Myle Ott, Anastasia Melnikov, Salvatore Candido, David Brooks, Geeta Chauhan, Benjamin Lee, Hsien-Hsin S. Lee, Bugra Akyildiz, Maximilian Balandat, Joe Spisak, Ravi Jain, Mike Rabbat, and Kim Hazelwood. 2022. Sustainable AI: Environmental Implications, Challenges and Opportunities. arXiv:2111.00364 [cs.LG] <https://arxiv.org/abs/2111.00364>
- [57] xAI. 2026. Colossus: The World’s Largest AI Supercomputer. <https://x.ai/colossus>
- [58] Jie You, Jae-Won Chung, and Mosharaf Chowdhury. 2023. Zeus: Understanding and Optimizing GPU Energy Consumption of DNN Training. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, Boston, MA, 119–139. <https://www.usenix.org/conference/nsdi23/presentation/you>
- [59] Junyeol Yu, Jongseok Kim, and Euseong Seo. 2023. Know Your Enemy To Save Cloud Energy: Energy-Performance Characterization of Machine Learning Serving. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. 842–854. doi:10.1109/HPCA56546.2023.10070943
- [60] Shan Yu, Jiarong Xing, Yifan Qiao, Mingyuan Ma, Yangmin Li, Yang Wang, Shuo Yang, Zhiqiang Xie, Shiyi Cao, Ke Bao, Ion Stoica, Harry Xu, and Ying Sheng. 2025. Prism: Unleashing GPU Sharing for Cost-Efficient Multi-LLM Serving. arXiv:2505.04021 [cs.DC] <https://arxiv.org/abs/2505.04021>
- [61] Dingyan Zhang, Haotian Wang, Yang Liu, Xingda Wei, Yizhou Shan, Rong Chen, and Haibo Chen. 2025. BLITZSCALE: fast and live large model autoscaling with O(1) host caching. In *Proceedings of the 19th USENIX Conference on Operating Systems Design and Implementation* (Boston, MA, USA) (*OSDI '25*). USENIX Association, USA, Article 16, 19 pages.
- [62] Yijia Zhang, Qiang Wang, Zhe Lin, Pengxiang Xu, and Bingqiang Wang. 2024. Improving GPU Energy Efficiency through an Application-transparent Frequency Scaling Policy with Performance Assurance. In *Proceedings of the Nineteenth European Conference on Computer Systems* (Athens, Greece) (*EuroSys '24*). Association for Computing Machinery, New York, NY, USA, 769–785. doi:10.1145/3627703.3629584

A Compute Infrastructure and Cluster Specifications

The measurements in this paper were collected on a large academic AI/HPC cluster. The cluster contains a heterogeneous NVIDIA GPU fleet spanning Ampere, Ada Lovelace, Hopper, and Blackwell generations. Importantly, the *installed* fleet is larger than the subset used in our telemetry study: not all GPUs could be included in the final dataset because some device generations had compatibility issues with our profiling stack (e.g., incomplete or unstable support in the telemetry tools used for continuous collection). Our analysis therefore uses the subset of devices for which profiling was reliable throughout the study period.

A.1 System Overview and Hardware Configuration

Table 3 summarizes the host environment, interconnect, and software stack. The cluster comprises 8,288 CPU cores and an installed fleet of 880 GPUs. Nodes run a production software

CPU Cores	8,288
Installed GPUs	880 NVIDIA GPUs
GPU Generations	Ampere, Ada Lovelace, Hopper, Blackwell
Interconnect	100 Gbps EDR InfiniBand (compute & storage) 1 GbE Ethernet (node management) 10 GbE Ethernet (external routing)
OS	AlmaLinux 9.5 (Teal Serval)
Kernel	5.14.0-503.40.1.el9_5.x86_64
Job Scheduler	Slurm 24.05.0 with QoS-aware preemption
NVIDIA Stack	Driver 575.51.03, CUDA 12.9, DCGM 3.3.9

Table 3. Overview of the academic GPU cluster used in this study.

GPU Model	Count	Set Power Limit
L40S	410	400 W
RTX A6000	208	300 W
RTX 6000 Ada Generation	58	300 W
L40	56	300 W
A100 80GB (PCIe)	48	300 W
RTX PRO 6000 (Blackwell)	40	600 W
A100 40GB (SXM4)	24	400 W
H100 (SXM5)	24	700 W
B200	8	1000 W
H200 (SXM)	4	700 W
Total	880	<i>Persistence Mode Enabled</i>

Table 4. Installed GPU fleet composition and default power limits reported by `nvidia-smi`.

stack based on AlmaLinux, Slurm, recent NVIDIA drivers, CUDA, and DCGM.

A.2 GPU Fleet Composition

Table 4 shows the installed GPU fleet and the corresponding default driver-enforced power limits reported by `nvidia-smi`. These counts describe the installed cluster hardware; as noted above, the profiled subset is smaller because telemetry collection was not uniformly supported across all device/tool combinations.

A.3 Network Topology

The cluster uses a dual-network design that separates management traffic from high-throughput data transfer. Compute nodes are equipped with NVIDIA/Mellanox MT4123 InfiniBand host channel adapters operating at 100 Gb/s for inter-node communication and storage access. Standard node management uses 1 GbE, while external routing uses 10 GbE. GPUDirect RDMA is not used for inter-GPU communication in this deployment.

A.4 GPU Configuration and Profiling Coverage

All GPUs operate in their factory-default configuration. NVIDIA Persistence Mode [27] is enabled, while Applications Clocks are left inactive. The driver therefore manages runtime DVFS and GPU Boost subject to the default hardware power limits, without manual clock locking or job-specific power caps.